

ภาคผนวก จ
ข้อรู้สัคัการแปลงฟาสท์ฟูรีเยร์

```
// ===== FFT.H =====//

#define FFT          -1
#define REV_FFT      1

// Radix 2 FFT calculation
// param = FFT      - performs FFT
// param = REV_FFT  - performs reverse FFT
// order = the order of FFT ( pow(2,order) calculated complex points )

void fft(double *real,double *imaginary,int order,int param);
```

```
// ===== FFT.CPP =====//

/*
Radix 2 FFT calculation
param = FFT    - performs FFT
param = REV_FFT - performs reverse FFT
order = the order of FFT ( pow(2,order) calculated complex points )
x = pointer to real
y = pointer to imaginary
*/

#include <math.h>
#include "FFT.H"

#ifndef M_PI
#define M_PI    3.14159265359
#endif

void fft( double *x,double *y,int order,int param ){
    unsigned int n,l,e,f,i,j,o1,j1,i1,k;
    double u,v,z,c,s,p,q,r,t,w,a;
    n=1u<<order;
    for( l = 1; l <= order; l++){
        u = 1.0;
        v = 0.0;
        e = 1u << ( order-l+1 );
        f = e/2;
        z = M_PI/f;
        c = cos(z);
        s = sin(z);
        if( param == FFT ) s = -s;
    }
}
```

```

for ( j = 1; j <= f; j++ ){
    for ( i = j; i <= n; i += e ){
        o = i+f-1;
        o1 = i-1;
        p = x[o1]+x[o];
        r = x[o1]-x[o];
        q = y[o1]+y[o];
        t = y[o1]-y[o];
        x[o] = r*u-t*v;
        y[o] = t*u+r*v;
        x[o1] = p;
        y[o1] = q;
    }
    w = u*c-v*s;
    v = v*c+u*s;
    u = w;
}

j = 1;
for ( i = 1; i < n; i++ ){
    if ( i < j ){
        j1 = j-1;
        i1 = i-1;
        p = x[j1];
        q = y[j1];
        x[j1] = x[i1];
        y[j1] = y[i1];
        x[i1] = p;
        y[i1] = q;
    }
    k = n/2;

```

```
        while ( k < j ){
            j = j-k;
            k = k/2;
        }
        j += k;
    }
    if ( param == FFT ) return;

    a = 1.0/n;
    for ( k = 0; k < n; k++ ){
        x[k] *= a;
        y[k] *= a;
    }
    return;
}
```